TILLÄMPAD ARTIFICIELL INTELLIGENS

F8 - BEHAVIOUR TREE

FEEDBACK PÅ KOD

- Strings!

- Varför är detta dåligt?

```
void Foo(const std::string& aString)
```

- Foo(”haha”) leder till en heap allocation
  - Om funktionen hade varit void Foo(const char* aString) så hade det varit gratis att call:a den både som Foo(”haha”) och Foo(aStdString).
    - Eftersom std::string convertas gratis till en c-string

- När ni pass:ar runt strängar: använd const char* !

INTRO

- Förra föreläsningen kollade vi på Decision Trees och State Machines.
  - Vi snackade snabbt också om kombinationer av dessa.

- Både Decision Trees och State Machines gör val som till slut resulterar i beslut.
  - Heal, Shoot, Wander, etc...

- AI:n har varit ganska enkel
  - Få inputs och få outpus
  - if (isClosestHostileInLOS) { // transition to stateShoot }

```
stateWander->SetTransitionState(stateShoot, [](const Actor& aActor)
{
    const bool isClosestHostileInLOS = aActor.GetIsClosestHostileInLOS();
    return isClosestHostileInLOS;
});
```

INTRO

- I moderna spel har spel-AI ofta mer komplexa val
  - Särskilt i AAA
- I större spel räcker inte Decision Trees och State Machines hela vägen.
  - Inget fel på teknikerna!
  - Bara lite för barebones för mer avancerad spel-AI
- Hur kommer det sig?

RECAP, DECISION TREES

- Bra:
  - Decision trees är väldigt lätta att skapa
  - Enkela att förstå, rent konceptuellt
  - Jämförelsevis enkelt att göra ett tool till AI-design för att bygga trädstrukturer

- Dåligt:
  - Exekveras från roten varje frame, kan bli många onödiga checkar.
  - Scale:ar inte så bra
    - 10 noder alright, men 50? 100? Big yikes.

RECAP, STATE MACHINES

- Bra:
  - Vi stannar i ett state tills något triggar en transition.
  - On average färre if-checkar än Decision Tree.

- Dåligt:
  - Om antalet states ökar så ökar ofta antalet transitions väldigt mycket.
    - Blir lätt spaghetti av att hålla reda på alla transitions.

RECAP, HSM

- Hierarchical State Machines (HSM) löser några av de problemen Decision Trees och State Machines har.

- Dvs en State Machine som kan ha andra system i sina states istället för bara beteenden.
  - T.ex. andra State Machines, eller Decision Trees

- Gör att vi kan ha få ”macro”-states, och därmed slippa massa transitions.
  - Men ändå ha mycket funktionalitet i states:en, som då är Decision Trees, t.ex.



BEHAVIOUR TREE

- Sammanfattningsvis är vårt problem scalability.
  - Både Decision Trees och State Machines har scale issues.
  - Det är därför de inte passar bra till mer avancerad AI.

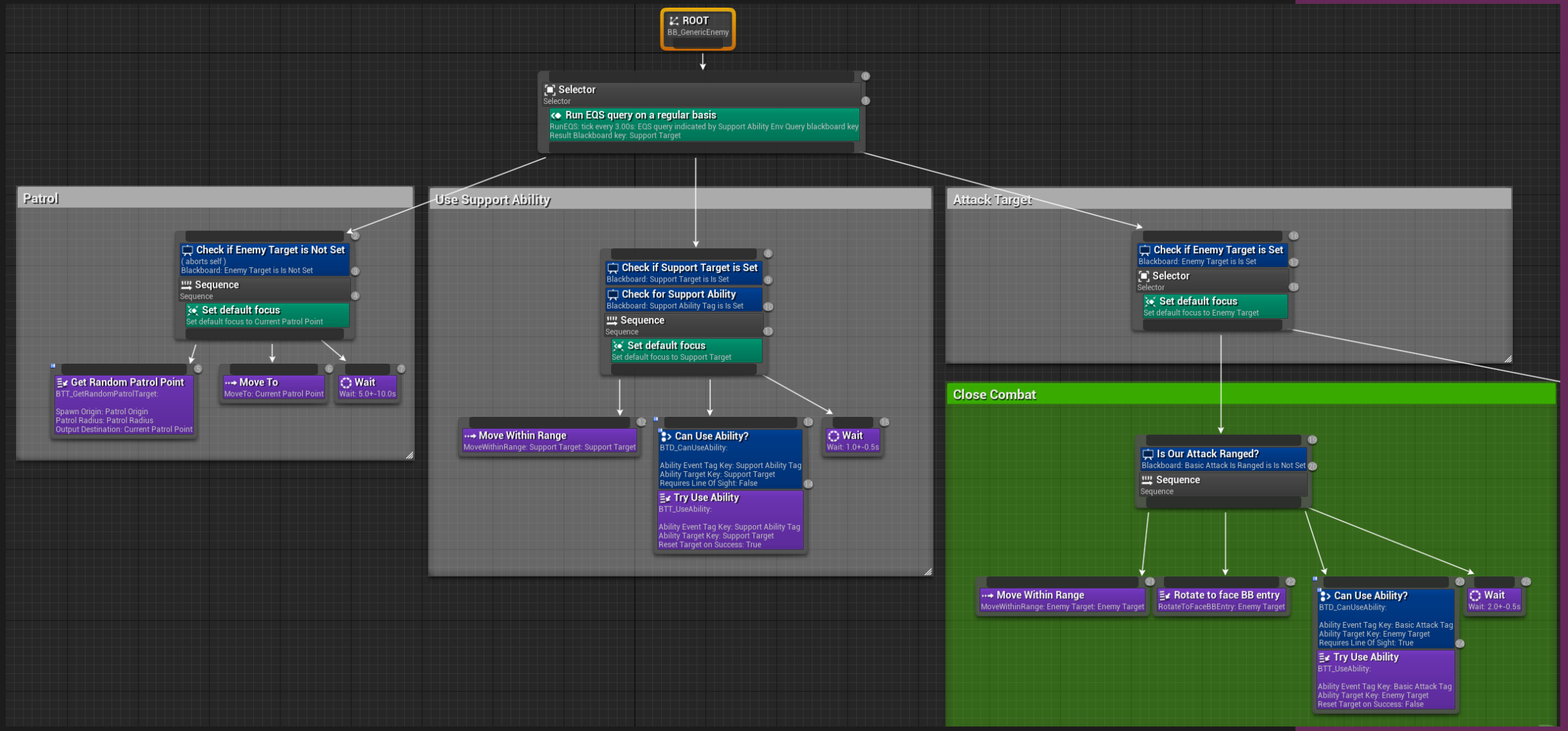
- Lösningen på alla våra problem är Behaviour Trees!
  - Minns ni när jag sa att NavMesh är da shit för navigation?
  - Behaviour Trees är da shit för decision making.

  - Men som alltid! Använd bara vid behov, dvs AI:n är lite mer invecklad.
    - Du tänder inte ett ljus med en flame thrower.

- Till skillnad från Decision Trees och State Machines så är Behaviour Trees en programmeringsteknik som är byggd för att användas just till AI.
  - De andra kan ju användas till annat än AI, t.ex. en State Machine för Game State.

- Till utseendet påminner Behaviour Trees om både Decision Trees och State Machines.
  - De är som en combo av båda. Best of both worlds.
  - Enda stora nackdelen är att de är mer komplexa. Fler termer att känna till.

BEHAVIOR TREE I UE5



BEHAVIOUR TREE

- I huvudsak finns det tre typer av noder
  - Composite
    - En nod som har x antal child nodes
    - Oftast Sequence eller Selector
      - (kommer snart till vad de är)
  - Decorator
    - Modifierar beteendet på en existerande nod
    - En vanlig decorator är en Invert
  - Leaf
    - Conditions och Actions
    - Dvs det som våra Decision Trees innehåller
- Finns många fler typer!
  - Men dessa är core-typerna
  - Unreal har massa custom-implementerade typer

BEHAVIOUR TREE

- Varje nod returnerar ett resultat som modifierar flödet. Vanligtvis implementerat som ett enum:
  - Running
  - Success
  - Failure

- Detta kontrollerar vad trädet exekverar

- Specifikt kontrollerar det våra flow control noder
  - Dvs Composite-noderna Sequence och Selector

COMPOSITE NODES

- Sequence
  - Exekverar sina child nodes i ordning tills någon returnerar Failure eller Running.
  - Om en child node returnerar Failure så gör Sequence-noden en early out och returnerar också Failure.
  - Om en child node returnerar Running så gör Sequence-noden en early out och returnerar också Running.
  - Om alla child nodes returnerar Success returnerar Sequence-noden Success.

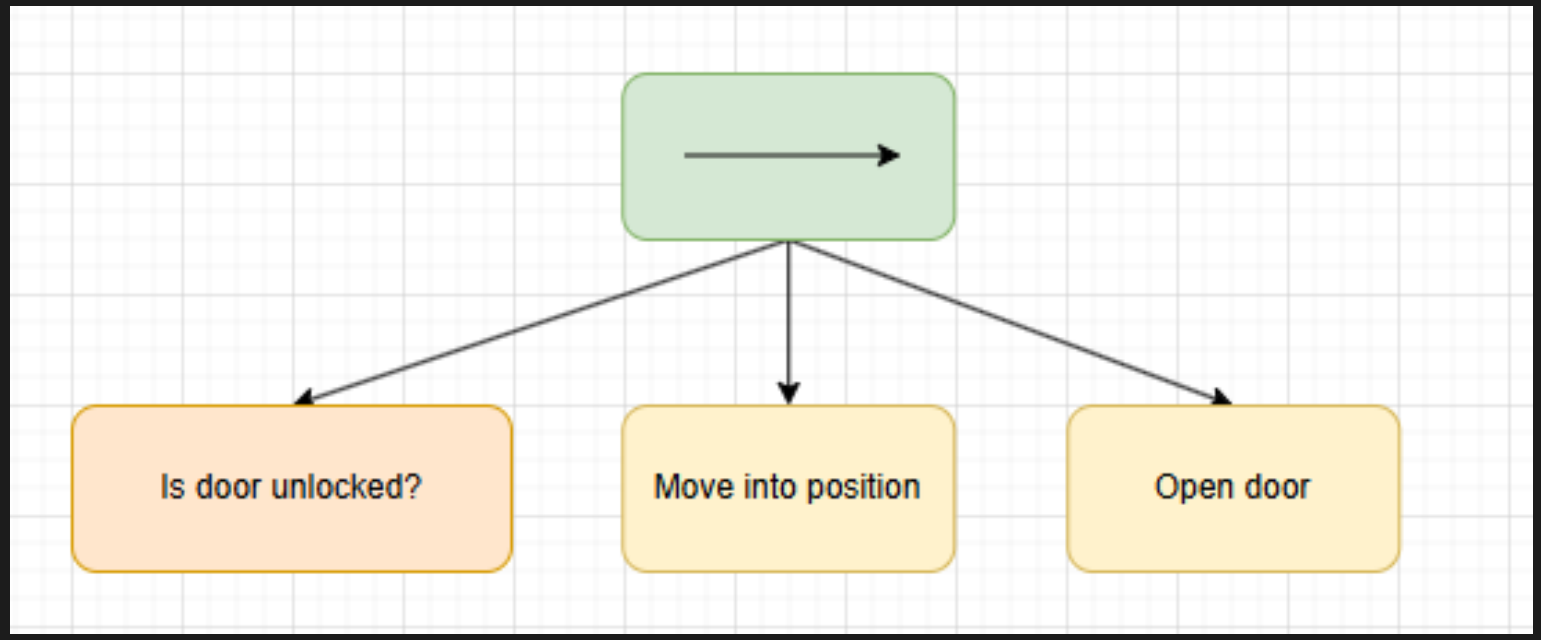
- Selector
  - Exekverar sina child nodes i ordning tills någon returnerar Success eller Running.
  - Om en child node returnerar Success så gör Selector-noden en early out och returnerar också Success.
  - Om en child node returnerar Running så gör Selector-noden en early out och returnerar också Running.
  - Om alla child nodes returnerar Failure returnerar Selector-noden Failure.

- Sequence vill att alla sina children ska returnera Success.
  - Lite som AND
- Selector nöjer sig med att en av sina children returnerar Success.
  - Lite som OR

SEQUENCE, EXEMPEL

- Brukar visualiseras med en pil.

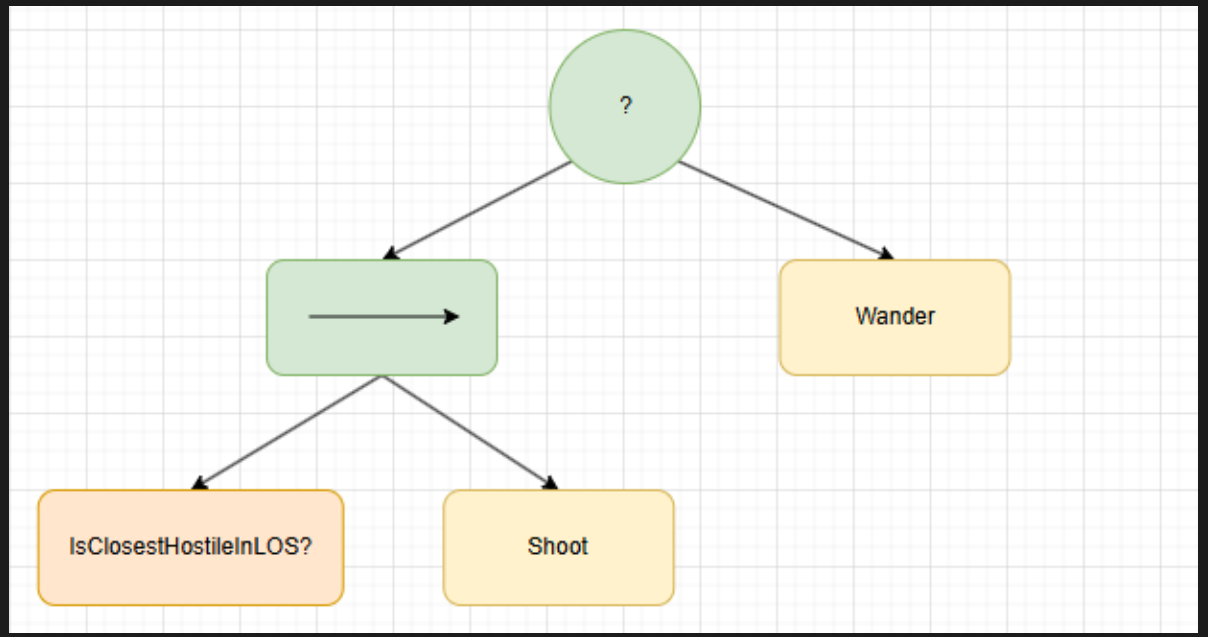
- Om dörren inte är unlocked så skippar sequence-noden att köra resten.



SELECTOR, EXEMPEL

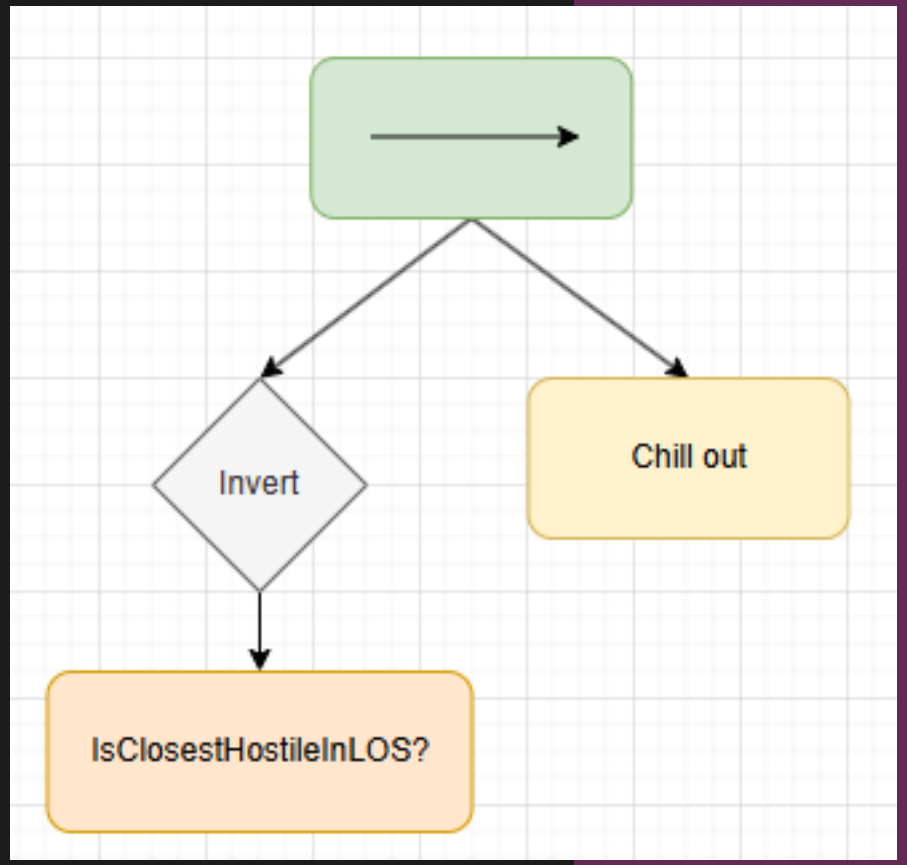
- Brukar visualiseras med ett frågetecken

- Detta är också ett exempel på hur sequence och selector kan kombineras.
  - Välj mellan ”LOS? -> Shoot” och Wander



DECORATOR, EXEMPEL

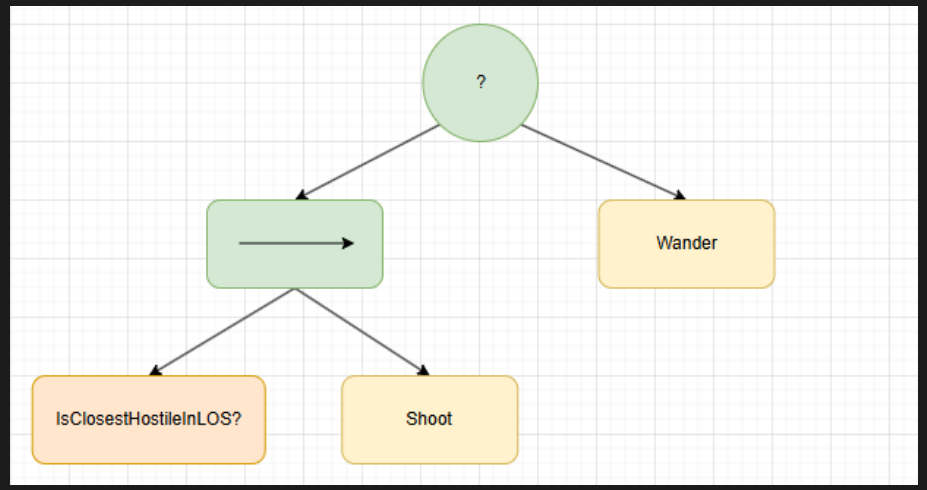
- Låter oss lätt ändra beteendet på en existerande nod.
  - Detta innebär att vi kan återanvända våra noder.
  - Istället för att ha IsClosestHostileInLOS och IsClosestHostileNotInLOS kan vi istället bara använda en Invert-decorator.
  - Finns hur många decorators som helst.
  - Andra som är vanliga är
    - Limit
      - Ser till att en action inte körs mer än x gånger.
    - Until Fail
      - Ser till att en action körs så länge den kan, tills den failar.



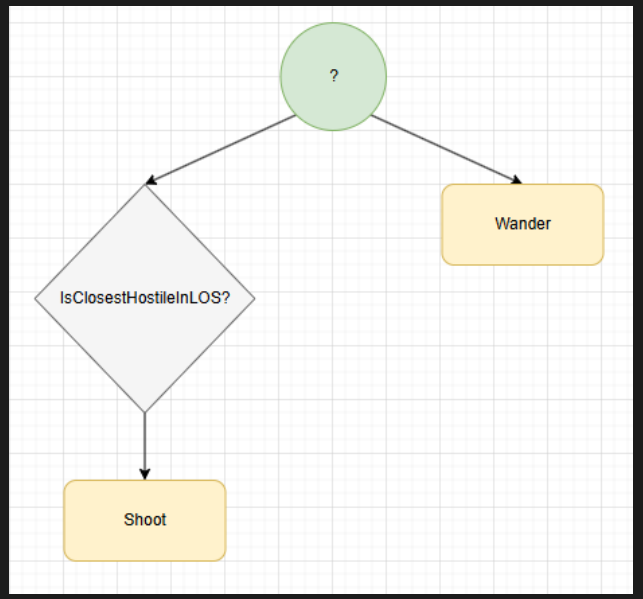
LEAF, EXEMPEL

- Sist har vi lövnoderna:
  - Lövnoderna är våra spelspecifika noder
  - De som innehåller alla conditions och actions

- En vanlig fråga
  - Varför använder vi inte decoratos som conditions?



VS



LEAF

- Conditions
  - Lövnoder som är Conditions är som if-satser.
  - Kontrollerar ifall vi vill köra andra noder.
  - Syns ofta först bland child nodes till en sequence.

- Actions
  - Ett behaviour
  - Wander, Shoot, SeekHP, osv…
  - Brukar ofta returnera Success, men kan returnera Failure eller Running berodende på implementation.
    - T.ex: lövnod Follow Path
    - Returnerar Running medan follow-beteendet exekveras.

SUBTREES

- En lövnod kan även anropa ett annat Behavior Tree.



- Exempel:
  - Vi har samma huvudträd för alla fiender
  - Vi har tre fiende-typer: melee, ranged och support.
  - Vi har en Leaf Node som heter Attack Target.
  - Vi kan ha tre olika Attack Trees som pluggas in i Attack Target.
    - Melee: går in i melee range och anfaller
    - Ranged: skjuter spelaren om den är i vision
    - Support: buffar en ally om den är i vision

- Nu ska vi kolla på kod-exempel! Men först...

PAUS



BEHAVIOUR TREE, KODEXEMPEL

- Förra föreläsningen introducerade vi
  - DecisionTreeActor och StateMachineActor
  - Hur ser BehaviourTreeActor ut?

```
class BehaviourTreeActor : public Actor
{
public:
    BehaviourTreeActor();

    void SetBehaviourTree(BehaviourTree* aBehaviourTree);
    void SetLeafNodeAction(const BT_LeafNodeAction* aLeafNodeAction);

private:
    void PreUpdate() override;
    void UpdateActorControllerData() override;
    void AddSteeringForce() override;
    void TriggerAction() override;

#if !IS_RETAIL_BUILD
    void UpdateDebugColor() override;
#endif // !IS_RETAIL_BUILD

    BehaviourTree* myBehaviourTree;
    const BT_LeafNodeAction* myLeafNodeAction;
};
```

- Istället för att vi har DT_ActionNode* eller State* så har vi nu BT_LeafNodeAction*.
- PreUpdate kör myBehaviourTree->Run().
- Men vad gör BehaviourTree::Run()?

BEHAVIOUR TREE, KODEXEMPEL

```
class BehaviourTree
{
public:
    BehaviourTree();

    void SetRoot(BT_Node* aRoot);
    void Run();
    void DeInit();

    Blackboard& GetBlackboard();

    void AddLeafNodeCondition(const BT_LeafNodeCondition* aLeafNodeCondition, bool aResultAtLastTick);

private:
    void CheckCurrentLeafNodeConditions();

    BT_Node* myRoot;
    Blackboard myBlackboard;
    bool myShouldTick;

    struct LeafNodeConditionAndResultAtLastTick
    {
        const BT_LeafNodeCondition* myLeafNodeCondition = nullptr;
        bool myResultAtLastTick = false;
    };

    std::vector<LeafNodeConditionAndResultAtLastTick> myCurrentLeafNodeConditionsAndResultsAtLastTick;
};
```

- Run()
  - Innebär att vi ”kör” trädet
  - Målet är att hitta en LeafNodeAction

- CurrentLeafNodeConditions
  - Logik för att inte behöva köra trädet från roten varje frame
  - (mer om detta snart)

BEHAVIOUR TREE, KODEXEMPEL

BT_Node.h

```
class BT_Node
{
public:
    BT_Node();
    virtual ~BT_Node() {}

    virtual void Init() {}
    virtual void DeInit() {}

    BT_NodeStatus Tick();

protected:
    virtual BT_NodeStatus Update() = 0;

private:
    BT_NodeStatus myStatus;
};
```

BT_Node.cpp

```
BT_Node::BT_Node()
    : myStatus(BT_NodeStatus::Invalid)
{
}

BT_NodeStatus BT_Node::Tick()
{
    if (myStatus != BT_NodeStatus::Running)
    {
        Init();
    }

    myStatus = Update();

    return myStatus;
}
```

- ”Tick” är bara terminologi för ”kör Init, om det behövs”.
  - Vissa noder kanske har ett state (dvs medlemsvariabler) som behöver Init:as vid en ny körning.
- Tick call:ar Update, som är där vi faktiskt kör nodens beteende.

BEHAVIOUR TREE, KODEXEMPEL

- Lite headers för core-noderna:

BT_CompositeNode

```
class BT_CompositeNode : public BT_Node
{
public:
    BT_CompositeNode();

    void DeInit() override;

    void AddChild(BT_Node* aChild);

protected:
    std::vector<BT_Node*> myChildren;
};
```

Sequence och Selector

```
class BT_SequenceNode : public BT_CompositeNode
{
public:
    BT_NodeStatus Update() override;
};
```

```
class BT_SelectorNode : public BT_CompositeNode
{
public:
    BT_NodeStatus Update() override;
};
```

BT_DecoratorNode

```
class BT_DecoratorNode : public BT_Node
{
public:
    BT_DecoratorNode();

    void DeInit() override;

    void SetChild(BT_Node* aChild);

protected:
    BT_Node* myChild;
};
```

BT_LeafNodeCondition och BT_LeafNodeAction

```
class BT_LeafNodeCondition : public BT_Node
{
public:
    BT_LeafNodeCondition(BehaviourTree* aBehaviourTree);

    BT_NodeStatus Update() override;

    virtual bool CheckCondition() const = 0;

protected:
    Blackboard* myBlackboard;

private:
    BehaviourTree* myBehaviourTree;
};
```

```
class BT_LeafNodeAction : public BT_Node
{
public:
    BT_LeafNodeAction(Blackboard* aBlackboard);

    virtual void UpdateActorControllerData(ActorControllerData& outActorControllerData) const;
    virtual HD_Vector2_f32 GetSteeringForce(const Actor& aActor) const;
    virtual void TriggerAction(Actor& aActor) const;

#if !IS_RETAIL_BUILD
    virtual Tga::Color DebugGetLeafNodeActionColor() const { return { 1.f, 1.f, 1.f, 1.f }; }
#endif // !IS_RETAIL_BUILD

protected:
    Blackboard* myBlackboard;
};
```

BEHAVIOUR TREE

- Hur får BehaviourTreeActor sin leaf node?
  - Dvs motsvarigheten till MakeDecision eller CheckStateTransition.
  - BehaviourTreeActor::PreUpdate call:ar Run på sitt Behaviour Tree.
    - Den lövnoden som vi hamnar i call:ar SetLeafNodeAction på actorn.
  - Från den lövnoden kan vi köra UpdateActorControllerData, AddSteeringForce och TriggerAction.
    - Precis som med en DT_ActionNode och ett State.

BEHAVIOUR TREE

- Hur gör vi så att vi inte behöver köra vårt behaviour tree från roten varje frame?
  - I BehaviourTree.cpp

```
struct LeafNodeConditionAndResultAtLastTick
{
    const BT_LeafNodeCondition* myLeafNodeCondition = nullptr;
    bool myResultAtLastTick = false;
};

std::vector<LeafNodeConditionAndResultAtLastTick> myCurrentLeafNodeConditionsAndResultsAtLastTick;
```

LeafNodes call:ar BehaviourTree::AddLeafNodeCondition och lägger till sig själva i denna std::vector

  - I BehaviourTree::Run gör jag detta:

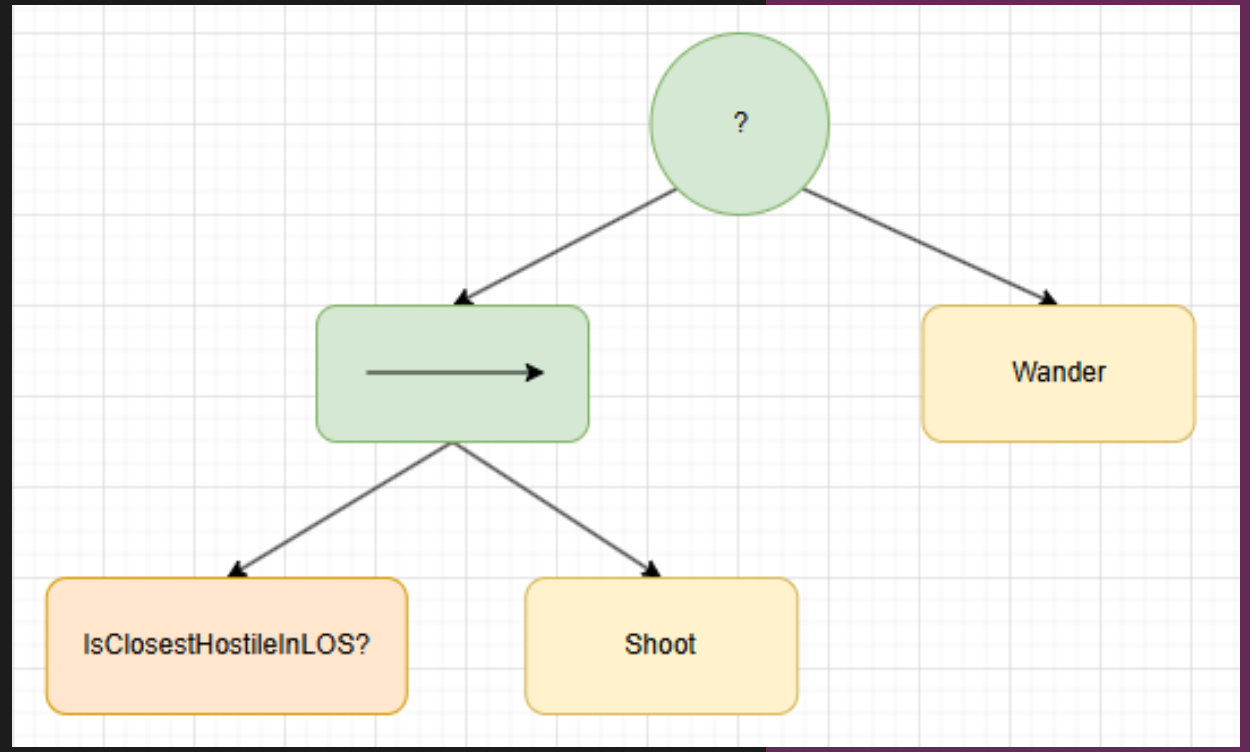
```
void BehaviourTree::Run()
{
    CheckCurrentLeafNodeConditions();

    if (myShouldTick)
    {
        myRoot->Tick();
        myShouldTick = false;
    }
}
```

  - CheckCurrentLeafNodeConditions loopar igenom myCurrentLeafNodeConditionsAndResultsAtLastTick och jämför leaf nodens nuvarande resultat med myResultAtLastTick.
    - Om något resultat skiljer sig från myResultAtLastTick så vet vi att vi bör tick:a från roten igen.

BEHAVIOUR TREE, EXEMPEL

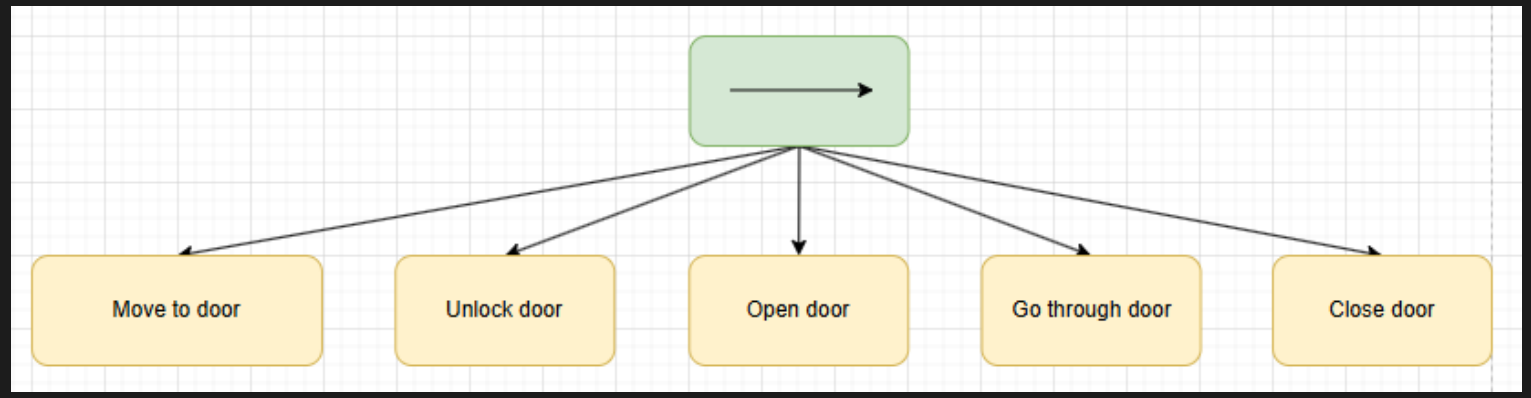
- 1. myBehaviourTree->Run()
  - Vid behov, Tick:ar myRoot
- 2. IsClosestHostileInLOS
  - Detta är en LeafNodeCondition
  - Call:ar AddLeafNodeCondition
    - Lägger till sig själv bland nuvarande conditions
- 3. Vi hamnar i Shoot eller Wander
  - Call:ar SetLeafNodeAction på vår Actor
- 4. När vi kör Run() nästa frame kollar vi om några conditions har ändrats.



BEHAVIOUR TREE

- Behavior Trees är dock inte felfria!
  - Det finns ett problem som ni kommer stöta på som inte alltid är lätt att lösa.

- En vakt har följande beteende:
  - Gå till en dörr -> lås upp dörr -> öppna dörr -> gå igenom -> stäng dörr.

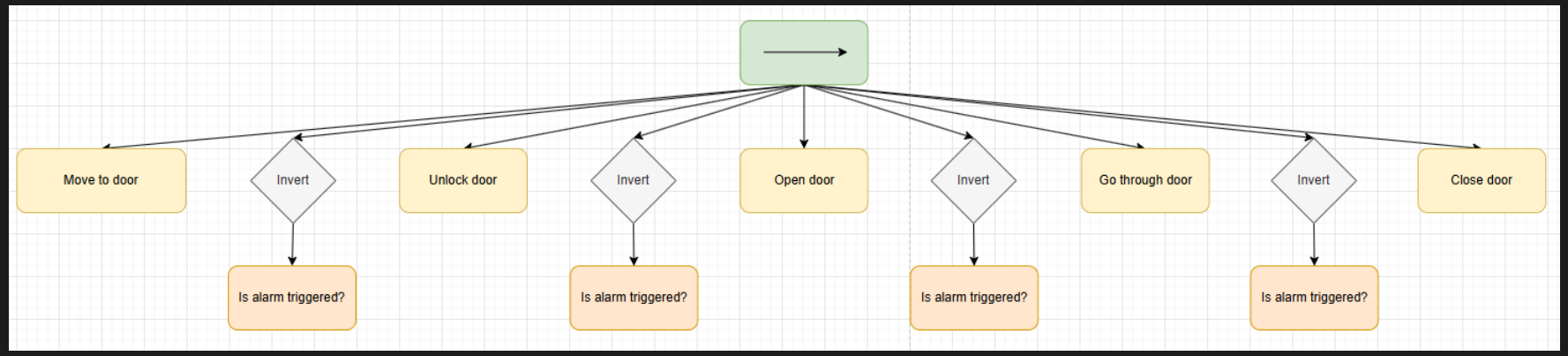


- Vad händer om spelaren triggar ett larm mitt i trädets körning?

- Det vi vill ska hända är att vakten avbryter sitt arbete och börjar reagera på spelaren.

BEHAVIOUR TREE

- Vi kan fylla trädet med massa condition leaf nodes som kollar om vi ska avbryta nuvarande beteende eller inte.



  - Det kan fungera om våra närliggande leaf nodes bara är saker som händer under ett fåtal frames.
  - Har vi en nod som returnerar Running kommer kollen att ske efteråt.
    - MoveToDoor t.ex. kommer säkerligen ofta köra ett par sekunder.

- Detta sättet kan alltså inte egentligen avbryta pågående beteenden. Så hur kan vi göra det?

BEHAVIOR TREE, PARALLEL

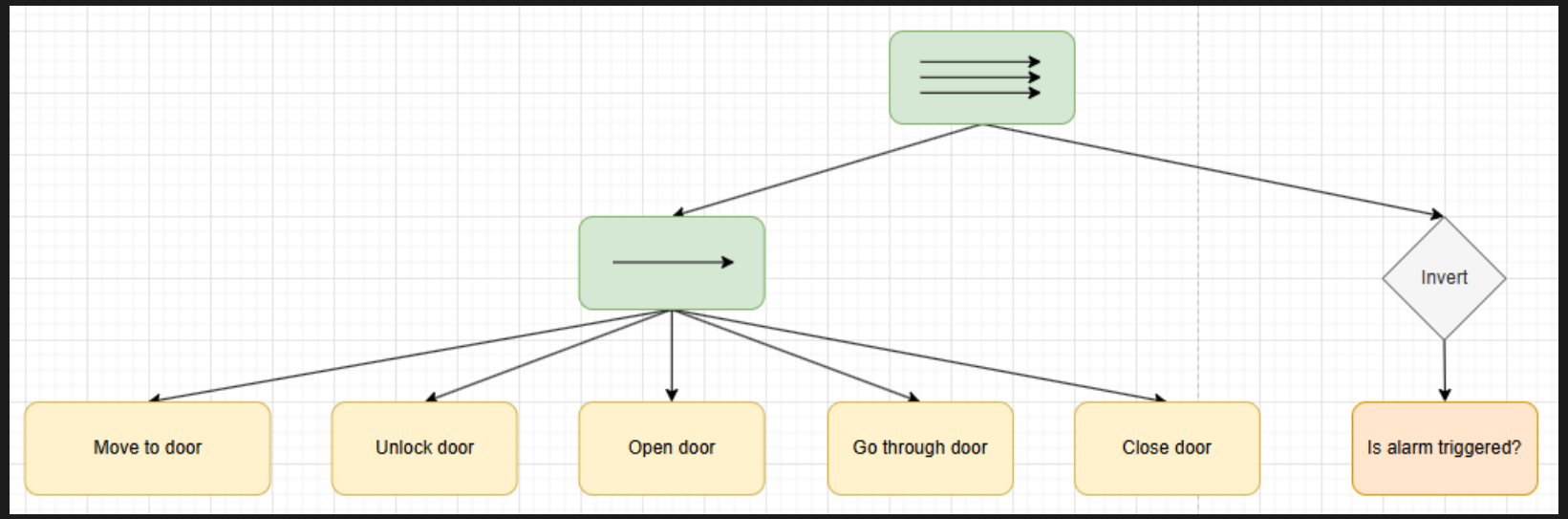
- Lösningen är en speciell Composite Node som heter Parallel.
  - Exekverar sina child nodes i ordning tills någon returnerar Failure.
  - Om en child node returnerar Failure så gör Parallel-noden en early out och returnerar också Failure.
  - Om alla child nodes returnerar Success returnerar Parallel-noden Success.
  - Om en eller flera child nodes returnerar Running returnerar Parallel-noden Running.

- Kan kännas väldigt lik Sequence
  - Viktigaste skillnaden är att Parallel fortsätter att köra nästa child även när ett child returnerar Running.

- En Parallel's child nodes behöver *inte* vara trådade
  - Vi hanterar dem som om de vore trådade, men rent praktiskt är de inte trådade.
  - Dvs den som bygger trädet kan tänka att de kör samtidigt.

BEHAVIOUR TREE, PARALLEL

- Nu ser det istället ut såhär!



- Mycket mindre kladd
- Och nu kan också beteenden som tar lång tid avbrytas, som ”Move to door”.

- Man kan se Parallel som en do { … } while (…)

AVSLUTNINGSVIS

- Finns lite gotchas med Behaviour Tree

- En grej är att eftersom ett Behaviour Tree sparar states i noderna (Success, Failure, Running), så är trädet kopplat till en actor-instans.

  - Dvs actors kan inte dela instanser av ett Behaviour Tree.
  - De kan säkert göra det, men inte vad jag vet.
  - Isf hade t.ex. actors behövt spara node states i Actor-klassen.
    - Låter rätt rörigt?

- Tidigare har vi kunnat komma undan med att actors delar DecisionTree/StateMachine-instans.
  - Funkar eftersom DecisionTree/StateMachine har fått all data de behöver för att köra från actorn.

AVSLUTNINGSVIS

- En annan grej är, hur får noder i ett Behaviour Tree tillgång till delad data? Och tillgång till actorn?

- I DecisionTree/StateMachine så skickar vi med en actor.

  - Vi brydde oss bara om GetSteeringForce och TriggerAction. Men nu har vi också en Update.
  - Vi kan skicka runt en actor-referens i Tick och Update.
  - Men enklaste imo är att BehaviourTree har en blackboard som den delar med sig till sina leaf nodes.

- Upp till er hur ni vill göra!

AVSLUTNINGSVIS, BLACKBOARD

- Osäker på hur en Blackboard ser ut?

```cpp
#pragma once

#include "BlackboardVariables.h"

class Actor;

class Blackboard
{
public:

    void SetBool(BlackboardVariable_Bools aBoolVariable, bool aValue);
    bool GetBool(BlackboardVariable_Bools aBoolVariable);

    void SetS32(BlackboardVariable_S32s aS32Variable, s32 aValue);
    bool GetS32(BlackboardVariable_S32s aS32Variable);

    void SetF32(BlackboardVariable_F32s aF32Variable, f32 aValue);
    bool GetF32(BlackboardVariable_F32s aF32Variable);

    void SetActor(BlackboardVariable_Actors aActorVariable, Actor* aValue);
    Actor* GetActor(BlackboardVariable_Actors aActorVariable);

private:
    std::unordered_map<BlackboardVariable_Bools, bool> myBools;
    std::unordered_map<BlackboardVariable_S32s, s32> myS32s;
    std::unordered_map<BlackboardVariable_F32s, f32> myF32s;
    std::unordered_map<BlackboardVariable_Actors, Actor*> myActors;
};
```

```cpp
enum class BlackboardVariable_Bools
{
    Count,
};

enum class BlackboardVariable_S32s
{
    Count,
};

enum class BlackboardVariable_F32s
{
    Count,
};

enum class BlackboardVariable_Actors
{
    OwnerActor,
    Count,
};
```

```cpp
void Blackboard::SetS32(BlackboardVariable_S32s aS32Variable, s32 aValue)
{
    myS32s[aS32Variable] = aValue;
}

bool Blackboard::GetS32(BlackboardVariable_S32s aS32Variable)
{
    if (myS32s.find(aS32Variable) == myS32s.end())
    {
        myS32s[aS32Variable] = 0;
    }

    return myS32s[aS32Variable];
}
```

AVSLUTNINGSVIS

- Behaviour Trees kan växa ganska mycket
  - Men de är enklare att ha en visuell översikt på dem än State Machines!
  - Eftersom de kan bli stora är det standard att spara en pekare/referens till det nuvarande beteendet som körs.

  - I mina exempel är det BT_LeafNodeAction-pekaren i BehaviourTreeActor.

RESURSER

- Förutom boken, mer läsning om Behaviour Trees:
  - https://www.gamedeveloper.com/programming/behavior-trees-for-ai-how-they-work
- Väldigt bra video för en översikt/introduktion till Behaviour Trees:
  - Samma kanal som videon om F.E.A.R
    - Har massvis med videos om olika AI-topics
  - https://www.youtube.com/watch?v=6VBCXvfNICM
- GDC Talk
  - AI Arborist: Proper Cultivation and Care for your Behaviour Trees
  - https://www.youtube.com/watch?v=Qq_xX1JCreI

KURSLITTERATUR

- Denna föreläsningen
  - Chapter 5 - Decision Making
    - 5.4 Behaviour Trees (s 338-374)

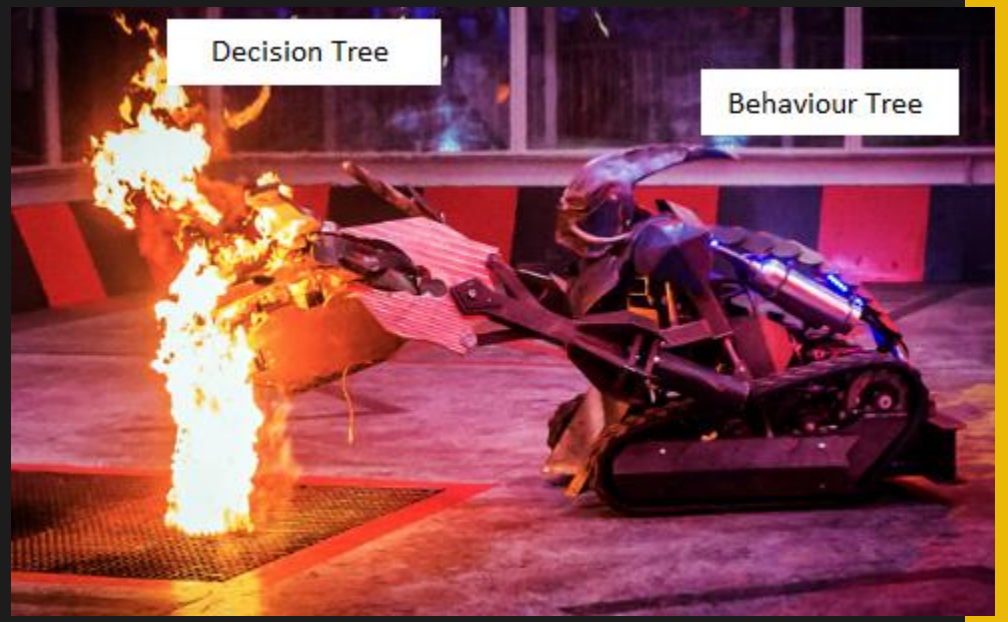
- Nästa föreläsning (om ni vill läsa lite om Learning AI)
  - Chapter 6 - Tactical and Strategic AI
    - 6.4 Coordinated Action (s 554-573)
  - Chapter 7 - Learning
    - 7.1 Learning Basics (s 575-579)
    - 7.2 Parameter Modification (s 579-592)
    - 7.3 Action Prediction (s 592-602)
  - Extra:
    - Chapter 10 - Execution Management
      - 10.1 Scheduling (s 803-820)
      - 10.2 Anytime Algorithms (s 820-821)

UPPGIFTEN

- Vi ska, you guessed it, implementera ett Behaviour Tree!

- Vi ska bygga vidare på Robot Wars
- En ny actor: Sir Killalot





UPPGIFTEN

- Veckans uppgift bygger på förra veckans uppgift.
- Ni ska lägga till en tredje actor, Sir Killalot, som styrs av ett Behaviour Tree.
- Sir Killalot ska ha samma beteende som de två andra.

- Ert Behaviour Tree ska innehålla minst en Selector Node och en Sequence Node.
- Conditions ska vara Leaf Nodes, inte Decorators.
- Ert Behaviour Tree ska på något sätt ”komma ihåg” var det senast var någonstans, och inte (som Decision Tree) exekveras från sin root varje frame.

- VG:
  - Ni ska lägga till en resurstyp som karaktärerna kan slåss om.
  - T.ex. ammo crates, power-ups, control-points som ger poäng.
  - Dessa kan implementeras som waypoints, och det ska finnas minst fyra st. De kan vara av samma typ, men det går också bra att implementera flera olika typer.
  - Alla tre actors ska kunna använda sig av waypointsen, alltså inte bara den som styrs med ett Behaviour Tree.

- VVVVVVVVVVVG:
  - Sir Killalot ska kunna plocka upp en annan actor och bära den till en burning fire pit of hell.
  - När den gör det så ska ”In Death - Is Death” av Meshuggah spelas:
    - https://www.youtube.com/watch?v=5CwwXxOwqVY&list=RD5CwwXxOwqVY&start_radio=1

DAGENS AI-FILMTIPS

- The Matrix, 1999 (Wachowskis)



FRÅGOR?

